

AI Finance Controller

Multi-source reconciliation — problem breakdown & solution design

1. Problem statement

Build an agent that closes one finance-ops loop across a 50+ record batch of synthetic data, reporting its match rate and the exceptions it could not resolve. Concretely: reconcile a bank/payment-gateway feed against an internal ledger, so every transaction ends in one of two states — matched, or flagged as an exception with a reason.

The judging bar: throughput (does it run on the full batch, not a cherry-picked example), measured accuracy (a real number against known-correct answers), and an honest exception list (unresolved cases surfaced clearly, not hidden or force-matched).

2. Why bank and ledger records never line up perfectly

The mismatch isn't artificial — it mirrors nine real causes finance teams deal with daily. Each belongs to one of four underlying categories, which is what your matching engine needs to detect.

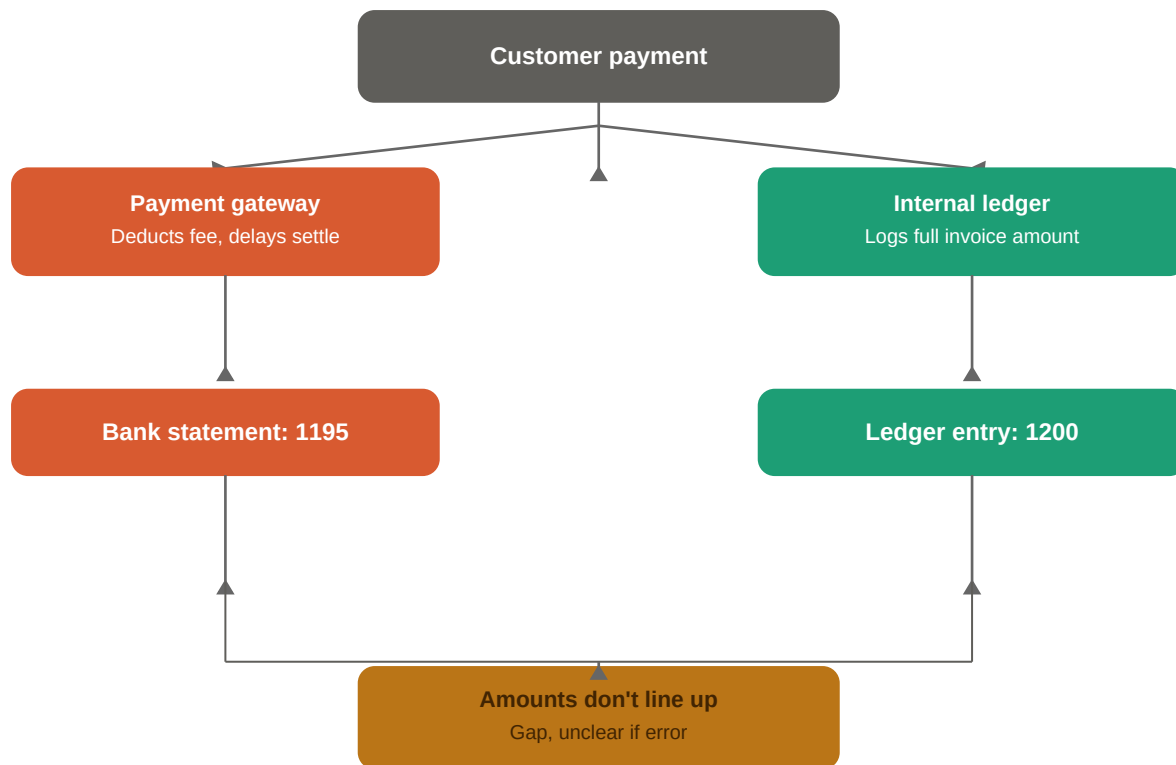
Cause	Category	What it looks like
Fees deducted before settlement	Amount	Ledger shows full invoice; bank shows amount minus processor fee.
Timing / settlement delay (T+1, T+2)	Timing	Sale recorded on one date, cash lands on a later date.
Batching	Structure (cardinality)	Processor bundles many small payments into one lump deposit.
Missing records	Missing	One side has no entry yet — unlogged internally, or still pending at the bank.
Duplicate entries	Structure (cardinality)	Same event manually re-keyed, creating two rows for one transaction.
Description / naming differences	Data quality	Generic processor text vs. internal invoice memo — no string overlap.
Currency / FX conversion	Amount	Invoice in USD vs. converted local-currency amount at day's rate.
Partial payments or refunds	Structure (cardinality)	One ledger entry maps to two bank entries, or vice versa.
Human error (typos)	Data quality	Manually keyed amount or date is simply wrong.

Category definitions:

- **Timing** — same transaction, different dates due to settlement lag.
- **Amount** — same transaction, different figures due to fees or FX conversion.
- **Structure (cardinality)** — not a 1:1 relationship: many-to-one (batching), one-to-many (partial/refund), or duplicated rows.
- **Data quality** — same transaction, unreliable or inconsistent text/values (naming, typos).
- **Missing** — an entry genuinely doesn't exist yet on one side.

This categorization matters because each category needs a different detection rule — you can't solve all nine causes with one string-match or one tolerance check.

Flowchart: how the mismatch happens



One real event splits into two independently recorded trails. A fee or delay is introduced on the way to one side, so the same transaction ends up with two numbers that both look correct on their own — that gap is the reconciliation problem.

3. Solution: a tiered matching pipeline

Instead of sending every record through an LLM (slow, non-deterministic, and expensive at 50+ rows), resolve records in increasing order of difficulty. Each tier handles the categories it's suited for, and only unresolved records fall through to the next tier.

Tier 1 — Exact match

Handles: Timing (none), Amount (none)

Match on identical amount + identical date (or ID). Resolves the clean majority instantly and deterministically.

Tier 2 — Fuzzy / rule-based match

Handles: Timing, Amount, Data quality

Amount within a fee/FX tolerance band, date within a settlement window (e.g. ± 2 days), and text similarity on description/memo. Also implement one-to-many / many-to-one grouping here to catch batching, partial payments, and refunds by summing candidate groups to the target amount.

Tier 3 — LLM on leftovers

Handles: Ambiguous cases across all categories

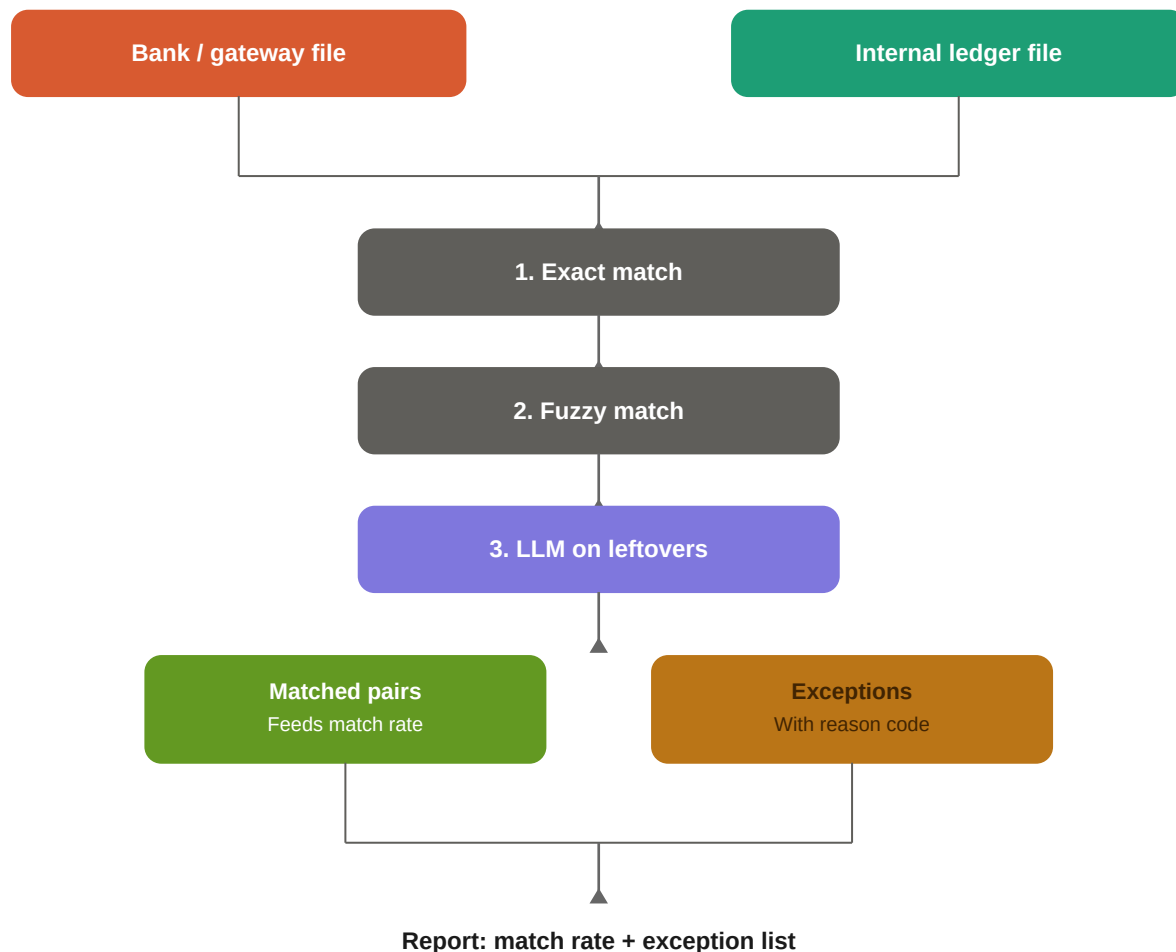
Only records tier 1 and 2 couldn't confidently resolve go to the LLM. It proposes a match with a plain-English justification, or explicitly returns 'unresolved' with a reason — it does not force a guess.

Tier 4 — Exception logging

Handles: Missing, unresolved ambiguous

Anything still unmatched becomes an exception row with a reason code (no candidate found, ambiguous multiple candidates, likely duplicate, etc.) instead of being hidden or dropped.

Flowchart: the matching pipeline



Records are filtered in increasing order of difficulty. Easy cases resolve instantly by rule; only genuinely ambiguous leftovers reach the LLM; anything still unclear becomes an honest exception with a reason instead of a forced match.

4. Step-by-step build order

- **Generate synthetic data with ground truth.** Write a script that creates a base set of true transactions, then derives a bank file and a ledger file from it, deliberately injecting each of the 9 causes above. Keep the original mapping — this is your answer key.
- **Build Tier 1 (exact match).** Straightforward equality check on amount + date/ID.
- **Build Tier 2 (fuzzy match).** Tolerance-based amount comparison, date-window comparison, string similarity for descriptions, and grouping logic for batched/partial amounts.
- **Build Tier 3 (LLM leftover resolution).** Send only the unresolved records with candidate context; require the model to return a structured decision (match / unresolved) with a reason.
- **Build the exception log and summary report.** Every record ends in matched or exception; compute match rate, and break down exceptions by reason code.
- **Score against ground truth.** Since you generated the data, compute precision/recall — this is the 'measured accuracy' judges are asked to look for.
- **Wire up the interface.** Upload two files, show live progress (throughput proof), display the match table and a prominent exceptions panel, and surface the summary numbers.

5. What the judges are actually checking

- **Throughput** — the full 50+ record batch runs live, not a hand-picked pair.
- **Measured accuracy** — a real precision/recall number against your own ground truth, shown on screen.
- **Honest exceptions** — unresolved records are visible with reasons, treated as a feature, not hidden.
- Avoid: a demo that only works on one cherry-picked match — that proves nothing per the brief.